



Universidad Carlos III
Grado Ingeniería Informática
Heurística y Optimización
Curso 2025-26

Fecha: 18/12/25 GRUPO: 84

Alumnos: **Xiya Ye** 100522159 - 100522159@alumnos.uc3m.es

Jorge Juan Sánchez Villalobos 100522377 - 100522377@alumnos.uc3m.es

Índice

1. Introducción.....	3
2. Parte 1 - BINAIRO.....	4
2.1 Modelización del problema.....	4
2.1.1 Variables.....	4
2.1.2 Dominios.....	4
2.1.3 Restricción 1: igualdad de blancos y negros en filas y columnas.....	5
2.1.4 Restricción 2: no más de dos discos iguales consecutivos.....	5
2.2 Implementación del modelo.....	5
2.2.1 Estructura general del programa:.....	6
2.2.2 Lectura y preparación de los datos.....	6
2.2.3 Creación de variables y dominios.....	6
2.2.4 Añadir restricciones.....	6
2.2.5 Resolución y salida.....	7
2.3 Resolución y análisis de los casos de prueba.....	7
3. Parte 2: Algoritmos de búsqueda.....	9
3.1 Modelización del problema.....	9
3.1.1 Representación del Grafo y Estados.....	9
3.1.2 Algoritmo de Búsqueda: A*.....	9
3.1.3 Definición de la heurística: Haversine.....	10
3.1.4 Diseño de Estructuras de Datos.....	10
3.2 Implementación del modelo.....	10
3.2.1 Representación del grafo.....	11
3.2.2 Estructuras de datos.....	11
3.2.3 Núcleo algorítmico.....	11
3.2.4 Script principal.....	12
4. Análisis de los resultados.....	13
4.1 Análisis de la Parte 1: CSP - Binario.....	13
4.2 Análisis de la Parte 2: Búsqueda Heurística.....	13
4.2.1 Calibración de la Heurística y Factor de Escala.....	13
4.2.2 Resultados Experimentales.....	14
4.2.3 Interpretación del Rendimiento.....	14
5. Conclusiones.....	15

1. Introducción

La presente memoria se encarga de documentar el desarrollo de la práctica de Satisfacción de Restricciones y Búsqueda Heurística. El objetivo principal de este trabajo es aplicar las técnicas fundamentales de la Inteligencia Artificial para la resolución de problemas complejos, dividiéndose en dos bloques: la Satisfacción de Restricciones (CSP) y la Búsqueda heurística en espacios de estados.

En la primera parte, se ha abordado la resolución del pasatiempo lógico Binairo. Para ello, hemos modelado el problema como un CSP (*Constraint Satisfaction Problem*), definiendo formalmente las variables, dominios y las restricciones estructurales del juego (igualdad de fichas por fila/columna y limitación de adyacencia). Hemos realizado la implementación en Python utilizando la librería *python-constraint*, comprobando la validez del modelo mediante diversos casos de prueba.

En la segunda parte, el trabajo se ha centrado en la búsqueda de rutas óptimas sobre mapas reales de carreteras de los Estados Unidos. El objetivo ha sido implementar estructuras de datos eficientes y desarrollar el algoritmo de búsqueda A*. Para garantizar la optimidad y mejorar el rendimiento frente a una búsqueda ciega, hemos diseñado una heurística admisible basada en la distancia Haversine, analizando el impacto de estas decisiones en el tiempo de ejecución y el número de nodos expandidos.

A lo largo de esta memoria, hemos detallado las decisiones de diseño tomadas, la modelización matemática de ambos problemas, la arquitectura del software desarrollado y un análisis crítico de los resultados experimentales obtenidos.

2. Parte 1 - BINAIRO

2.1 Modelización del problema

El problema de esta primera parte consiste en resolver un puzzle tipo Binairo definido sobre un tablero de tamaño $n \times n$. En la instancia inicial del problema, cada casilla puede contener un disco negro (X), un disco blanco (O) o estar vacía (en este caso, el “vacío” se representa como un punto “.”). A partir de esta disposición, el objetivo es completar el tablero cumpliendo las siguientes condiciones impuestas por el juego:

- En cada fila y en cada columna debe haber el mismo número de discos blancos que de discos negros.
- No puede haber más de dos discos consecutivos del mismo color ni en filas ni en columnas.
- Todas las casillas deben quedar ocupadas en la solución final.
- Las casillas que ya traen un disco fijado en el fichero de entrada no se pueden modificar.

Para modelar esta parte como un problema de satisfacción de restricciones (CSP), se ha seguido el enfoque que se describe a continuación.

2.1.1 Variables

Para definir a las variables, hemos “aplanado” el tablero usando una lista de longitud n^2 ($n \times n$). Cada casillas del tablero (i, j) se identifica con un índice k , dado por:

$$k = i * n + j, \quad \text{donde } 0 \leq i, j < n$$

Para cada índice k , definimos una variable en el CSP. Así, el conjunto de variables resultante es:

$$\{x_0, x_1, \dots, x_{n^2-1}\}.$$

2.1.2 Dominios

Cada variable x_k representa el color de la casilla correspondiente. Para resolver el problema se ha usado una codificación binaria:

- Si la casilla contiene un disco blanco (O), su dominio será $\{0\}$.
- Si la casilla contiene un disco negro (X), su dominio será $\{1\}$.
- Finalmente, si la casilla está vacía (.), su dominio será $\{0, 1\}$.

De este modo, las casillas del tablero se incorporan directamente restringiendo el dominio de las variables.

2.1.3 Restricción 1: igualdad de blancos y negros en filas y columnas

Si nos fijamos en una línea del tablero, esta tiene n casillas, que representamos con variables $x_0, \dots, x_n$.

Usamos la codificación 0 = blanco y 1 = negro, por lo que la suma $\sum_{i=1}^n x_i$ coincide con el número de discos negros de esa fila, mientras que el número de discos blancos es $n - \sum_{i=1}^n x_i$.

De este modo, la condición de “mismo número de discos blancos y de negros” se define como:

$$\#negros = \#blancos \Leftrightarrow \sum x_i = n - \sum x_i \Leftrightarrow 2\sum x_i = n$$

Esta restricción se implementa en la función `mitad_blancos`, que recibe como argumentos los valores de una fila (o columna) y comprueba exactamente el cumplimiento de esa igualdad. Una consecuencia de esta restricción es que solo puede satisfacerse si n es par: con un número impar, el modelo no producirá soluciones válidas.

2.1.4 Restricción 2: no más de dos discos iguales consecutivos

La segunda regla impuesta por el juego nos dice que no puede haber más de dos discos consecutivos con el mismo color: es decir, que no haya más de dos discos del mismo color en disposición horizontal o vertical. Si consideramos una tupla de tres variables (a, b, c) correspondiente a tres casillas consecutivas en una fila o columna, la restricción se expresa como:

$$\neg(a = b = c)$$

En el modelo, se recorre el tablero generando todas las tuplas horizontales y verticales posibles, y se aplica sobre cada una de ellas una función `no_tres_seguidos(a, b, c)`, que devuelve falso exactamente cuando los tres valores son iguales.

2.2 Implementación del modelo

La implementación del problema lo hemos realizado en Python usando la librería *python-constraint*. El programa principal se encuentra en el fichero *parte-1.py*, ejecutando:

```
.parte-1.py fichero_entrada fichero_salida
```

2.2.1 Estructura general del programa:

- *ruta_absoluta*: función auxiliar que construye rutas absolutas a partir de los nombres de los ficheros dado por línea de comandos.
- *leer_entrada*: se encarga de leer el fichero de entrada y construir una lista (matriz) con todos los símbolos del tablero.
- *construir_modelo*: recibe una matriz y el tamaño n y construye el objeto *constraint.Problem* con todas las variables y restricciones.
- *mitad_blancos* y *no_tres_seguidos*: funciones que implementan las restricciones lógicas del modelo.
- *mostrar_instancia*: imprime por pantalla la instancia original en formato de tablero.
- *escribir_salida*: genera el fichero con la instancia original y una de las soluciones encontradas.

En la función *main* se gestionan los parámetros de entrada, se lee el archivo de entrada, se construye el modelo, se resuelve el CSP y finalmente se muestra el resultado.

2.2.2 Lectura y preparación de los datos

En *leer_entrada(fichero_entrada)* se leen las líneas del fichero, usamos *strip()* para eliminar los saltos de línea, y vamos añadiendo los símbolos leídos a una lista. A continuación, calculamos n , haciendo la raíz cuadrada de la longitud total de la lista (número de símbolos leídos). Para asegurarnos de que el número total de casillas corresponde a un tablero $n \times n$, comprobamos si $n \times n$ es igual a la longitud de la lista, en caso de que no sea así, significa que el número de elementos no forma una matriz cuadrada.

2.2.3 Creación de variables y dominios

En *construir_modelo(matriz, n)* se crea un objeto *constraint.Problem()* y se recorre la lista matriz. Para cada posición i :

- si $\text{matriz}[i] == \text{'O'}$ se añade la variable i con dominio $[0]$.
- si $\text{matriz}[i] == \text{'X'}$ se añade la variable i con dominio $[1]$.
- si $\text{matriz}[i] == \text{'.'}$ se añade la variable i con dominio $[0, 1]$.

De esta forma se añade la información de la instancia original dentro del propio dominio de las variables.

2.2.4 Añadir restricciones

- Restricción 1: igualdad de blancos y negros en filas y columnas

- Para cada fila i , hemos construido la lista de variables, cogiendo elementos de la lista matriz de n en n , y con eso añadimos la restricción *mitad_blanco*s sobre esa lista nueva.
- Para cada columna j , hemos hecho lo mismo, pero cogiendo los valores de la columna.
- Restricción 2: Recorremos el tablero para generar todas las tuplas de tres elementos de manera horizontal y verticales, añadiendo la restricción *no_tres_seguidos* a cada tupla.

Finalmente, *construir_modelo* devuelve el objeto problema.

2.2.5 Resolución y salida

En *main*, una vez construido el modelo, usamos la función *getSolutions()* para obtener la lista de soluciones, donde cada solución es un diccionario que asigna a cada índice un valor 0 o 1. Por eso, a la hora de mostrarlo por pantalla o escribirlo en el fichero de salida, tenemos en cuenta que hay que traducir los 0's a O y los 1's a X.

- *mostrar_instancia(matriz, n)* muestra por pantalla la instancia original.
- *escribir_salida(salida, soluciones, matriz, n)* genera el fichero de salida. Escribe primero la instancia original como tablero con bordes, y en el caso de que haya soluciones, la escribimos también, pero si no hay, escribimos un mensaje de error.

2.3 Resolución y análisis de los casos de prueba

- A. **Caso Básico:** Funcionamiento general, restricciones básicas de filas/columnas y no-consecutivos. (prueba-1-1.in)

Instancia Original

```
+---+---+---+---+---+
| | X | | | X |
+---+---+---+---+---+
| | | X | | O |
+---+---+---+---+---+
| | O | | X | |
+---+---+---+---+---+
| O | | X | | |
+---+---+---+---+---+
| | X | | O | |
+---+---+---+---+---+
| | | | | |
+---+---+---+---+---+
```

Resultado

```
+---+---+---+---+---+
| O | X | X | O | X |
+---+---+---+---+---+
| O | X | X | O | X | O |
+---+---+---+---+---+
| X | O | O | X | X | O |
+---+---+---+---+---+
| O | O | X | X | O | X |
+---+---+---+---+---+
| X | X | O | O | X | O |
+---+---+---+---+---+
| X | O | O | X | O | X |
+---+---+---+---+---+
```

Métricas

- Soluciones encontradas: 11
- Estado: Válido

B. **Tablero pequeño:** Se adapta a diferentes dimensiones n sin romper la lógica. (prueba-1-2.in)

Instancia Original

```
+---+---+---+
| X | X |   |
+---+---+---+
|   | X | X |
+---+---+---+
|   |   | X | X |
+---+---+---+
| X |   |   | X |
+---+---+---+
```

Resultado

```
+---+---+---+
| X | X | 0 | 0 |
+---+---+---+
| 0 | X | X | 0 |
+---+---+---+
| 0 | 0 | X | X |
+---+---+---+
| X | 0 | 0 | X |
+---+---+---+
```

Métricas

- Soluciones encontradas: 1
- Estado: Válido

C. **Tablero vacío:** Probamos el modelo de restricciones. (prueba-1-3.in)

Instancia Original

```
+---+---+---+---+
| | | | | | |
+---+---+---+---+
| | | | | | |
+---+---+---+---+
| | | | | | |
+---+---+---+---+
| | | | | | |
+---+---+---+---+
| | | | | | |
+---+---+---+---+
| | | | | | |
+---+---+---+---+
```

Resultado

```
+---+---+---+---+
| X | 0 | X | X | 0 | 0 |
+---+---+---+---+
| 0 | X | 0 | 0 | X | X |
+---+---+---+---+
| X | 0 | X | X | 0 | 0 |
+---+---+---+---+
| X | 0 | X | X | 0 | 0 |
+---+---+---+---+
| 0 | X | 0 | 0 | X | X |
+---+---+---+---+
| 0 | X | 0 | 0 | X | X |
+---+---+---+---+
```

Métricas

- Soluciones encontradas: 11222
- Estado: Válido

D. **Caso imposible I:** Probamos que la restricción de “El numero de discos blancos y negros en cada fila, y en cada columna, debe ser el mismo ” cumple. (prueba-1-4.in)

Resultado

```
No se han encontrado soluciones.
```

Métricas

- Soluciones encontradas: 0
- Estado: Infactible (correcto)

E. **Caso imposible II:** Probamos que la restricción de “No es posible disponer mas de dos discos del mismo color consecutivamente en una fila o columna ” cumple. (prueba-1-5.in)

Resultado

```
No se han encontrado soluciones.
```

Métricas

- Soluciones encontradas: 0
- Estado: Infactible (correcto)

3. Parte 2: Algoritmos de búsqueda

3.1 Modelización del problema

El problema de la Parte 2 consiste en encontrar el camino de coste mínimo entre dos puntos geográficos en un mapa de carreteras real. Para resolverlo, hemos modelado el sistema como un problema de búsqueda en un espacio de estados.

3.1.1 Representación del Grafo y Estados

Hemos definido el problema mediante un grafo dirigido $G = (V, E)$, donde:

- V (Vértices): Cada estado del problema corresponde a una intersección o punto del mapa, identificado por un ID numérico único. Cada vértice tiene asociado una coordenada geográfica (longitud y latitud) que utilizamos para la heurística.
- E (Arcos): Las transiciones entre estados que vienen dadas por los tramos de carretera. Cada arco (u, v) que representa la distancia física en metros.
- Costes: Como los fichero de entrada proporciona las distancias en metros como números enteros, el coste del camino $g(n)$ lo hemos modelado también como un valor entero.

3.1.2 Algoritmo de Búsqueda: A*

Con el objetivo de obtener la ruta óptima minimizando el número de nodos expandidos, hemos elegido el algoritmo A*. Este algoritmo evalúa los nodos basándose en la función de coste:

$$f(n) = g(n) + h(n)$$

Hemos elegido usar este algoritmo por su eficiencia a la hora de calcular la ruta óptima, condición que nos asegura al utilizar la distancia Haversine como una heurística admisible ($h(n) \leq h^*(n)$). Hemos considerado que esta estrategia es más adecuada que una búsqueda no informada, como el algoritmo de Dijkstra, ya que nos permite dirigir la exploración hacia el objetivo y reducir de forma significativa tanto el espacio de búsqueda como el número de nodos que deben expandirse.

Del mismo modo, hemos decidido descartar alternativas iterativas como IDA*, porque su comportamiento resulta poco eficiente en dominios con costes reales no unitarios, provocando un elevado número de re-expansiones. En nuestro caso, hemos optado por priorizar el tiempo de ejecución frente al consumo de memoria, por lo que el uso de una Lista Cerrada en A* nos ha parecido que es la solución más adecuada para gestionar los ciclos del mapa y evitar exploraciones redundantes.

3.1.3 Definición de la heurística: Haversine

Como los nodos tienen coordenadas geográficas sobre la superficie terrestre, hemos implementado la Distancia Haversine como función heurística $h(n)$. Esta fórmula calcula la distancia del arco de círculo máximo entre el nodo actual y el destino.

- Admisibilidad: La distancia en línea recta siempre es menor o igual que la distancia real por carretera. Por tanto, $h(n) \leq h^*(n)$, garantizando que A* encuentre la solución óptima.
- Adaptación al modelo: Para mantener la consistencia con los costes enteros del grafo y optimizar la estructura de datos, el valor resultante de la fórmula Haversine lo truncamos a entero, usando la función *floor*. Esto mantiene la admisibilidad ($\text{floor}(h) \leq h \leq h^*$) y permite trabajar con prioridades enteras.

3.1.4 Diseño de Estructuras de Datos

Somos conscientes de que la eficiencia del algoritmo A* depende en gran medida de cómo se gestionan los conjuntos de nodos abiertos y cerrados. Por este motivo, hemos abordado el diseño de las estructuras de datos como una decisión clave, teniendo en cuenta tanto las características del problema como los objetivos de rendimiento.

Para el control de los nodos ya visitados, decidimos implementar la Lista Cerrada mediante una *table hash*, utilizando un conjunto (*set*). Esta elección nos permite comprobar si un nodo ya ha sido expandido en tiempo constante $O(1)$. Hemos considerado y descartado el uso de listas o arrays, ya que habrían implicado búsquedas lineales $O(n)$, poco adecuadas para un grafo que supera los 300.000 nodos.

En cuanto a la Lista Abierta, hemos optado por una cola de prioridad basada en Dial's Bucketing. Dado que los costes de los arcos y la heurística son valores enteros, las prioridades $f(n)$ también lo son, lo que nos permite organizar los nodos en cubos indexados por su valor de prioridad mediante un diccionario de listas. Esta estructura nos permite realizar inserciones y extracciones del nodo con menor prioridad en tiempo amortizado $O(1)$, mejorando el rendimiento frente a un montículo binario tradicional con complejidad $O(\log n)$. Teniendo en cuenta el elevado número de inserciones que realiza el algoritmo, hemos considerado que esta decisión se alinea con nuestro objetivo de priorizar el tiempo de ejecución frente al consumo de memoria.

3.2 Implementación del modelo

La solución de la segunda parte también se ha desarrollado en Python, estructurando el código en diferentes módulos para favorecer la legibilidad y el mantenimiento del software. La arquitectura se divide en cuatro componentes: la definición del grafo (*grafo.py*), las estructuras de datos auxiliares

(*abierta.py* y *cerrada.py*), el núcleo algorítmico (*algoritmo.py*) y el script principal de gestión de entrada/salida (*parte-2.py*).

3.2.1 Representación del grafo

La clase Grafo es la encargada de la persistencia en memoria del mapa. Dada la magnitud de los ficheros de entrada, se ha priorizado la eficiencia en el acceso:

- Para la carga de datos, se han implementado los métodos *_cargar_coordenadas* y *_cargar_arcos*, que procesan los ficheros *.co* y *.gr* respectivamente.
- Respecto a la estructura interna, se utiliza un diccionario de Python para la lista de adyacencia, donde cada clave es un ID de nodo y el valor es una lista de tuplas (*vecino, coste*). Esto permite acceder a los vecinos de un nodo en tiempo $O(1)$ en promedio. Las coordenadas se almacenan en otro diccionario para permitir consultas rápidas durante el cálculo de la heurística.
- En cuanto a la optimización de memoria, se convierten explícitamente los costes y los IDs a enteros para reducir la huella de memoria respecto al uso de cadenas o flotantes innecesarios.

3.2.2 Estructuras de datos

Para soportar el algoritmo A^* , se han implementado dos estructuras clave descritas en la fase de modelización:

1. Lista Cerrada (*cerrada.py*): se ha implementado mediante la clase *Cerrada*, que encapsula un conjunto (*set*) nativo de Python. Esto garantiza que la verificación de pertenencia y la inserción se realicen en tiempo constante $O(1)$, algo fundamental para evitar ciclos y re-expansiones redundantes.
2. Lista Abierta (*abierta.py*): se ha implementado la clase *Abierta* siguiendo la estrategia de *Dial's Bucketing*. Se utiliza un diccionario donde las claves son los valores de prioridad $f(n)$ y los valores son listas de nodos. Para optimizar la extracción del mínimo, se mantiene un puntero *min_key* que rastrea el coste mínimo actual, evitando búsquedas lineales sobre todas las prioridades existentes. Esto permite operaciones de inserción y extracción muy eficientes.

3.2.3 Núcleo algorítmico

La clase *Algoritmo* encapsula la lógica de búsqueda. Su método principal, *buscar*, implementa el bucle estándar de A^* :

- El método *heuristica* calcula la distancia Haversine entre el nodo actual y el destino utilizando fórmulas trigonométricas sobre las coordenadas geográficas obtenidas del grafo. El resultado se trunca a entero para ser compatible con la estructura de cubos de la lista abierta.

- Se mantienen diccionarios para $g(n)$ y para los punteros a los padres, permitiendo la reconstrucción del camino óptimo una vez alcanzado el destino mediante el método *reconstruir_camino*.
- La implementación también permite, mediante el parámetro *heuristica_cero*, anular la función heurística (haciendo $h(n) = 0$), lo que convierte el algoritmo A^* en el algoritmo de Dijkstra. Esto es esencial para la fase de análisis comparativo.

3.2.4 Script principal

Por su parte, el fichero *parte-2.py* orquesta la ejecución del programa. Se encarga de:

1. Validar y procesar los argumentos de línea de comandos: vértices de inicio/fin, mapa y fichero de salida.
2. Instanciar el grafo y ejecutar el algoritmo midiendo el tiempo de CPU.
3. Generar el fichero de salida con el formato especificado: nodo, coste del arco y siguiente nodo.
4. Mostrar por pantalla las estadísticas requeridas: número de vértices/arcos cargados, coste total, tiempo de ejecución y tasa de nodos expandidos por segundo.

4. Análisis de los resultados

En esta sección se detallan los resultados experimentales obtenidos, analizando el rendimiento de las soluciones implementadas y respondiendo a las cuestiones de complejidad planteadas en el enunciado.

4.1 Análisis de la Parte 1: CSP - Binario

En relación con la modelización del problema de satisfacción de restricciones, el tamaño del problema se define en función de las dimensiones del tablero $n \times n$.

Se han definido un total de n^2 variables, correspondientes a cada una de las casillas del tablero. Así mismo, el modelo incluye restricciones de igualdad de conteo para cada una de las n filas y n columnas, así como restricciones de “no tres consecutivos” aplicadas a todas las tuplas horizontales y verticales posibles. Esto asegura que el espacio de búsqueda se reduzca drásticamente antes de probar asignaciones.

Las pruebas realizadas en la sección 2.3 demuestran que el solucionador *python-constraint* gestiona eficazmente tableros de tamaño estándar, aunque el tiempo de resolución crece exponencialmente con n .

4.2 Análisis de la Parte 2: Búsqueda Heurística

Para evaluar la eficiencia del algoritmo A^* implementado con la distancia Haversine, hemos realizado una comparativa frente a una búsqueda de fuerza bruta, como lo es el algoritmo de Dijkstra. Se han seleccionado tres instancias de diferente magnitud sobre el mapa completo de los Estados Unidos (USA).

4.2.1 Calibración de la Heurística y Factor de Escala

Uno de los hallazgos técnicos más relevantes durante la fase experimentación fue la discrepancia de unidades entre el grafo de entrada y la distancia física real. Inicialmente, observamos que el algoritmo A^* expandía un número de nodos prácticamente idéntico al de Dijkstra, sin ofrecer mejoras en el tiempo de ejecución.

Tras analizar los costes acumulados $g(n)$, determinamos que los archivos de mapas proporcionados utilizan una unidad de medida aproximada al decímetro, resultando en valores numéricos unas 10 veces superiores a la distancia en metros calculada por la fórmula de Haversine. Esta diferencia de magnitud provocaba que el valor de la heurística $h(n)$ fuera despreciable frente al coste acumulado, anulando el efecto de “guía” hacia el objetivo y degenerando el comportamiento de A^* en una búsqueda ciega equivalente a Dijkstra.

Para corregir esto manteniendo la garantía de optimalidad, se introdujo un factor de escala $K = 9.5$ en la función heurística: este factor asegura que la estimación nunca supere el coste real, preservando así la condición de admisibilidad y garantizando que el algoritmo encuentre siempre la ruta de coste mínimo.

4.2.2 Resultados Experimentales

La siguiente tabla resume los resultados obtenidos ejecutando ambos algoritmos sobre las mismas instancias (es decir, mismo origen y destino). El coste de la solución es idéntico en ambos casos, lo que valida la admisibilidad de la heurística implementada.

TIPO DE CASO	ORIGEN - DESTINO	ALGORITMO	NODOS EXPANDIDOS	TIEMPO (s)	COSTE
Corto (~3,03 km)	23.164.715 - 23.157.876	Dijkstra	219	0,0068 s	39.354
		A^*	161	0,0064 s	39.354
Medio (~2.240 km)	23.164.715 - 9.448.481	Dijkstra	6.927.139	20,8316 s	27.040.668
		A^*	2.385.315	14,5577 s	27.040.668
Largo (~4.480 km)	23.164.715 - 3.944.181	Dijkstra	23.945.205	112,7977 s	52.705.891
		A^*	15.521.306	126,1087 s	52.705.891

4.2.3 Interpretación del Rendimiento

Los resultados obtenidos en los tres casos de prueba evidencia cómo la eficiencia de A^* escala con la complejidad del problema.

En distancias cortas, la reducción en el número de nodos expandidos es porcentualmente menor y el tiempo de ejecución es similar al de Dijkstra. Esto se debe a que, en trayectos breves, el coste computacional fijo de calcular la heurística, el cual incluye operaciones trigonométricas, tiene un impacto proporcional mayor frente al ahorro de expansiones.

En distancias largas, la ventaja de A^* se vuelve drástica. La heurística permite podar inmensas regiones del grafo que Dijkstra exploraría innecesariamente. Al dirigir la exploración exclusivamente hacia el destino, A^* logra reducir en gran medida el número de expansiones necesarias para encontrar la ruta con coste óptimo.

No obstante, en nuestro último caso de prueba, observamos que el algoritmo de Dijkstra llegó a ser hasta 14 segundos más rápido que A^* . Este fenómeno se explica por el balance entre la poda y el coste por nodo:

- Cada nodo procesado por A^* requiere ejecutar la fórmula de Haversine, la cual incluye operaciones costosas para el intérprete de Python. Por contra, Dijkstra solo realiza sumas y comparaciones de enteros, que son mucho más rápidas.
- En mapas con rutas obligadas, la heurística no puede podar suficientes ramas laterales para compensar su propio coste de cálculo. En este caso en particular, se escogieron dos de los puntos más extremos del mapa (Norte y Sur). Ya que por la propia naturaleza del escenario los algoritmos están obligados a ir al Sur, no hay suficientes ramas para podar para compensar dicho coste.
- Aunque A^* es algorítmicamente más eficiente, en implementaciones de alto nivel como Python, la fuerza bruta aritmética de Dijkstra puede resultar más veloz en tiempo de reloj para distancias extremas donde la poda no es masiva.

5. Conclusiones

La realización de esta práctica nos ha permitido profundizar en dos campos muy diversos.

En la primera parte, hemos comprobado cómo una correcta modelización matemática es determinante para la resolución de problemas combinatorios. La definición precisa de variables y restricciones permitió delegar la complejidad de la búsqueda en el motor de *python-constraint*. Aprendimos que, aunque la programación con restricciones es declarativa y potente, su escalabilidad depende críticamente de cómo se definen los dominios y las reglas para podar el espacio de búsqueda lo antes posible.

En la segunda parte, el desarrollo del algoritmo A^* nos supuso todo un reto. Observamos que la implementación eficiente de la lista abierta mediante *Dial's Bucketing* y la lista cerrada con tablas *hash* fue crucial, y pudimos comprobar que la complejidad teórica del algoritmo no sirve de nada si las operaciones subyacentes de inserción y consulta no son $O(1)$ en la práctica. También aprendimos que una heurística no calibrada correctamente inutiliza a A^* , degradándolo a Dijkstra, y así mismo pudimos comprobar de primera mano cómo expandir menos nodos no siempre equivale a menor tiempo de ejecución si se dan las circunstancias específicas ya mencionadas anteriormente.

En conclusión, este trabajo nos ha demostrado que la elección del algoritmo y la heurística no es una decisión aislada, sino que depende de la naturaleza de los datos, la escala del problema y las limitaciones del entorno de ejecución.